

SQL Server & T-SQL Avançado

Fundamentos da Programação, Ambiente de Desenvolvimento e Otimização de Otimização de Desempenho.

query.sql - SSMS

```
> Conectado a localhost\SQLEXPRESS  
-- Conexão estabelecida
```

```
SELECT @@VERSION AS VersaoSQLServer
```

RESULT

✓ 1 row · 00:00:00

3

BLOCOS

24

SLIDES

110'

DURAÇÃO

01

Agenda da Aula

Três grandes blocos cobrindo desde a visão geral até otimização avançada.

01 / 03



Visão Geral do SQL Server e T-SQL

- >O que é o SQL Server
- >Arquitetura e Componentes
- >Edições e Recursos
- >Introdução ao T-SQL
- >T-SQL Avançado: Categorias e Recursos

 ~ 25 MIN

02 / 03



Fundamentos da Programação no SQL Server Server

- >Variáveis e Controle de Fluxo
- >Batches e Estrutura de Blocos
- >Stored Procedures
- >Funções Definidas pelo Usuário
- >Triggers
- >Tratamento de Erros e Transações
- >CTEs e Consultas Avançadas

 ~ 50 MIN

03 / 03



Ambiente, Monitoramento e Otimização

- >SQL Server Management Studio
- >Ferramentas de Monitoramento
- >Índices e Estatísticas
- >Plano de Execução
- >Boas Práticas de Desempenho

 ~ 35 MIN

BLOCO 01 / 03

01

Visão Geral do SQL Server e T-SQL Avançado

O que é, como funciona e por que dominar o T-SQL é decisivo para qualquer profissional de profissional de dados.

- 01** → O que é o SQL Server
- 02** → Arquitetura e Componentes
- 03** → Edições e Recursos
- 04** → Introdução ao T-SQL
- 05** → T-SQL Avançado — Categorias e Recursos

1 CONCEITO FUNDAMENTAL

O que é o SQL Server

Microsoft SQL Server é um Sistema de Gerenciamento de Banco de Dados Relacional (SGBD) desenvolvido pela Microsoft, projetado para armazenar, recuperar e gerenciar dados estruturados em escala empresarial. Utiliza a linguagem **SQL** (Structured Query Language) como interface padrão, estendida com o estendida com o dialeto proprietário **T-SQL** (Transact-SQL).

Lançado em **1989** como parceria Microsoft-Sybase, o SQL Server evoluiu até se tornar uma das principais plataformas de banco de dados do mercado, presente em sistemas críticos de bancos, e-commerces, analíticas. Está disponível em múltiplas edições, desde a **Express** gratuita até a **Enterprise** de alta

FICHA TÉCNICA

v.2025

FABRICANTE	Microsoft
LANÇAMENTO	1989 (v1.0)
VERSÃO ATUAL	SQL Server 2022 (16.x)
LINGUAGEM	T-SQL (Transact-SQL)
LICENCIAMENTO	Proprietário + Express Free
PLATAFORMAS	Windows · Linux · Docker · Azure SQL
DB ENGINES	#32805

Disponível também como PaaS: Azure SQL Database & SQL SQL Managed Instance.

1989

LANÇAMENTO

v16.x

SQL 2022

#3

DB ENGINES

◆ ARQUITETURA INTERNA

Arquitetura e Componentes do SQL Server

Camadas que trabalham em conjunto para processar, armazenar e proteger seus dados.



01

Relational Engine

Processa consultas T-SQL: parsing, otimização, execução. execução. Contém o Query Optimizer que gera planos de planos de execução eficientes.



02

Storage Engine

Gerencia a leitura/escrita física de dados em páginas de 8KB, organizadas em arquivos .mdf, .ldf, .ndf e filegroups.



03

Buffer Pool

Cache em memória RAM que mantém páginas de dados recentes para reduzir I/O em disco. Crítico para desempenho.



04

Transaction Log

Registra todas as transações para garantir ACID. Suporta rollback e recuperação pós-falha (crash recovery).



05

SQL Server Services

Serviços auxiliares: SQL Server Agent (jobs), Integration Services (ETL), Analysis Services (OLAP), Reporting Services.



06

Network Protocols

Comunicação via TCP/IP, Named Pipes, Shared Memory. Cada conexão usa TDS (Tabular Data Stream).

⚙ Cada componente é tunável via `sp_configure` e DMVs (`sys.dm_*`)

↔ EDITIONS MATRIX

Edições e Recursos do SQL Server

Escolha da edição impacta custo, escala e recursos disponíveis.

DEV = ENTERPRISE

RECURSO	Express FREE	Standard PAGO	Enterprise PAGO ALTO	Developer DEV
\$ CUSTO	Gratuito	Pago	Pago (alto)	Gratuito (dev)
🏠 USO TÍPICO	Aprendizado, apps pequenas	PMEs, LOB apps	Crítico, alta escala	Desenvolvimento/teste
🏠 LIMITE DE RAM	1.410 MB	128 GB	Ilimitado (OS)	Ilimitado (OS)
🏠 TAMANHO DB	10 GB	524 PB	524 PB	524 PB
🌟 CORES (CPU)	1 socket / 4 cores	Limitado por licença	Ilimitado	Ilimitado
↔ ALWAYS ON	—	Básico	Avançado	Avançado
⚡ HIGH PERF WORKLOAD	—	Parcial	Total	Total
👉 RECOMENDADO PARA	Estudantes	Produção média	Produção grande	Laboratório

◆ LINGUAGEM

Introdução ao T-SQL

T-SQL (Transact-SQL) é uma extensão proprietária da Microsoft para a linguagem **SQL** padrão ANSI/ISO, acrescentando controle de fluxo, variáveis, tratamento de erros, funções internas e construções procedurais que transformam o SQL declarativo em uma linguagem de programação completa dentro do banco de dados.

→ DIFERENÇAS EM RELAÇÃO AO SQL ANSI:

- → Controle de fluxo (IF, WHILE, CASE, TRY/CATCH)
- → Variáveis locais (@var e globais @@var)
- → Stored Procedures, Funções, Triggers
- → Tratamento de transações explícito (BEGIN TRAN / COMMIT / ROLLBACK)
- → Funções internas (GETDATE(), @@ROWCOUNT, ERROR_MESSAGE())
- → Batches separados por GO

● ● ● exemplo_tsq1.sql

◆ T-SQL

```
-- Exemplo: T-SQL procedural
DECLARE @LimiteCredito DECIMAL(10, 2) 5000.00
DECLARE @ClienteId INT 42

IF EXISTS
    SELECT 1 FROM
        WHERE
            @ClienteId
            AND
                @LimiteCredito

BEGIN
    PRINT 'Cliente excedeu o limite'
    RAISERROR('Limite ultrapassado', 16, 1)
END
ELSE
    PRINT 'Compra aprovada'

GO
```

> 16 linhas · batch único

✓ syntax ok

T-SQL Avançado — Categorias e Recursos

Os subconjuntos que compõem o dialeto Transact-SQL.



DDL

DATA DEFINITION

Define estrutura: CREATE, ALTER, DROP. DROP. Aplica-se a tabelas, views, procedures, índices.

EXEMPLO

CREATE TABLE, ALTER VIEW, DROP INDEX



DML

DATA MANIPULATION

Manipula dados: SELECT, INSERT, UPDATE, UPDATE, DELETE, MERGE.

EXEMPLOS

UPDATE ... OUTPUT, MERGE INTO



DCL

DATA CONTROL

Permissões: GRANT, REVOKE, DENY. Controla segurança e acessos.

EXEMPLO

GRANT SELECT ON Cliente TO app_user



DTL

TRANSACTION CONTROL

Transações: BEGIN TRAN, COMMIT, ROLLBACK, SAVE TRAN. Garante ACID. ACID.

EXEMPLO

BEGIN TRY ... BEGIN TRAN



PROCEDURAL

CONTROLE DE FLUXO

IF/ELSE, WHILE, CASE, GOTO, RETURN, RETURN, WAITFOR. Lógica procedural. procedural.

EXEMPLO

WHILE @i < 10 BEGIN ... END



PROGRAMABILIDADE

REUSABLE LOGIC

Stored Procedures, Functions, Triggers. Triggers. Reutilização de lógica.

EXEMPLOS

CREATE PROCEDURE, CREATE FUNCTION



ROBUSTEZ

TRATAMENTO DE ERROS

TRY/CATCH, THROW, RAISERROR, @@ERROR. Robustez.

EXEMPLO

BEGIN TRY ... BEGIN CATCH ... END CATCH



CONSULT AS AVANÇAD AS

ANALYTICAL

CTEs, Window Functions, PIVOT, CROSS CROSS APPLY, recursão.

EXEMPLOS

WITH cte AS (...), ROW_NUMBER() OVER(...)

BLOCO 02 / 03

02

Fundamentos da Programação no SQL Server

Da declaração de variáveis às construções procedurais avançadas — o coração do T-SQL.

- 10 → Variáveis e Controle de Fluxo
- 11 → Batches e Estrutura de Blocos
- 12 → Stored Procedures
- 13 → Funções Definidas pelo Usuário
- 14 → Triggers
- 15 → Tratamento de Erros e Transações
- 16 → CTEs e Consultas Avançadas

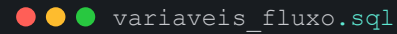
PROGRAMAÇÃO BÁSICA

Variáveis e Controle de Fluxo

T-SQL permite declarar variáveis locais (@nome) com tipos SQL Server, e controlar fluxo com construções procedurais.

CONSTRUÇÕES

DECLARE @var TIPO	declaração
SET @var = valor	atribuição simples
SELECT @var = coluna	atribuição por consulta
IF ... BEGIN ... END	condicional
WHILE ... BEGIN ... END	laço
CASE WHEN ... THEN ... END	expressão condicional
BEGIN ... END	agrupa blocos
RETURN	sai de procedure
WAITFOR DELAY / TIME	aguarda

variaveis_fluxo.sql

```
-- Exemplo: cálculo de desconto progressivo
DECLARE @Valor DECIMAL(10, 2) 1500.00
DECLARE @Desconto DECIMAL(5, 2) 0
DECLARE @ClienteId INT 1024

IF @Valor > 1000
BEGIN
    SET @Desconto = 15.0
    PRINT 'Cliente VIP — desconto de 15%'
END
ELSE IF @Valor > 500
BEGIN
    SET @Desconto = 10.0
    PRINT 'Desconto padrão de 10%'
END
ELSE
    SET @Desconto = 5.0

-- Laço WHILE para simular parcelamento
DECLARE @Parcela INT 1
WHILE @Parcela <= 3
BEGIN
    PRINT 'Parcela ' + CAST(@Parcela AS VARCHAR)
    PRINT ' — valor: R$ '
    PRINT '          ' + CAST(@Valor * (1 - @Desconto / 100) AS VARCHAR)
    SET @Parcela = @Parcela + 1
END
GO
```

✓ Execução: 1 lote · 0 erros

T-SQL · SSMS 19

ORGANIZAÇÃO DE CÓDIGO

Batches e Estrutura de Blocos

Um batch é um conjunto de instruções T-SQL enviadas ao SQL Server para execução como uma unidade atômica. O separador GO marca o fim de um batch — um batch — mas atenção: GO não é uma instrução T-SQL, é um delimitador delimitador reconhecido apenas por ferramentas como SSMS e sqlcmd.

REGRAS IMPORTANTES

- ❏ Variáveis locais existem apenas dentro do batch em que foram declaradas.
- ! CREATE PROCEDURE / FUNCTION / TRIGGER / VIEW deve ser a única instrução instrução no batch.
- ✦ GO aceita um contador: **GO 100** executa o batch 100 vezes.
- ⊖ Algumas instruções não podem ser combinadas no mesmo batch.
- ⊗ Erro de sintaxe em um batch cancela **TODO** o batch, mas batches seguintes continuam.

batches_separados_por_GO.sql

```
-- Batch 1: declaração e uso de variável
DECLARE @Msg VARCHAR(50)    'Olá, banco de dados!'
PRINT @Msg
GO
-- A partir daqui, @Msg não existe mais

-- Batch 2: criação de procedure (sozinha no batch)
CREATE PROCEDURE
    @DataInicial DATE
AS
BEGIN
    SET NOCOUNT ON
    SELECT          SUM          AS
    FROM
    WHERE           @DataInicial
    GROUP BY

END
GO
-- Batch 3: uso da procedure
EXEC              '2026-01-01'
GO 1
```

✓ Execução: 3 lotes · 0 erros

SSMS 19 · sqlcmd

PROGRAMABILIDADE

Stored Procedures

Procedimentos armazenados — reutilização, performance e segurança.

CARACTERÍSTICAS

- Código T-SQL armazenado no banco, executado por chamada `EXEC`.
- Compilação e plano de execução em cache — ganho de performance.
- Parâmetros de entrada (`@param`) e de saída (`OUTPUT`).
- Controle de permissões granular (`GRANT EXECUTE`).
- Reduz tráfego de rede — apenas parâmetros trafegam.

QUANDO USAR

- Lógica de negócio reutilizável
- Operações transacionais completas
- Validações e auditoria centralizada

sp_parametro_output.sql

```
-- SP com parâmetro de saída (OUTPUT)
CREATE PROCEDURE
    @ClienteId INT
    @Total DECIMAL(10, 2) OUTPUT
AS
BEGIN
    SET NOCOUNT ON
    SELECT @Total = SUM
        FROM
            WHERE
                @ClienteId
    IF @Total IS NULL
        SET @Total = 0
END
GO

-- Chamada da SP capturando o OUTPUT
DECLARE @Resultado DECIMAL(10, 2)
EXEC
    @ClienteId 1024
    @Total @Resultado OUTPUT
PRINT 'Total: R$ ' + CAST(@Resultado AS VARCHAR)
```

✓ Procedure criada · OUTPUT capturado

dbo.usp_ObterTotalCliente

EXEMPLO PRÁTICO

SP com Validação, Transação e Log

Procedimento completo para transferência bancária entre contas, com validação, transação ACID, log de auditoria e auditoria e tratamento de erro.

PONTOS DIDÁTICOS

- ✂ TRY/CATCH para robustez
- ✦ BEGIN TRAN / COMMIT / ROLLBACK
- ✓ Validação do saldo via EXISTS
- ☰ Log em tabela de auditoria
- ✦ Retorno via OUTPUT

CRÍTICA

⚠ USP_TRANSFERIRFUNDOS.SQL ·
FINANCEIRO

usp_TransferirFundos.sql

```
CREATE PROCEDURE
    @DeConta INT
    @ParaConta INT
    @Valor DECIMAL(10, 2)
    @Sucesso BIT OUTPUT
AS
BEGIN
    SET NOCOUNT ON
    SET @Sucesso = 0

    BEGIN TRY
        BEGIN TRAN
        -- Validação de saldo suficiente
        IF NOT EXISTS
            SELECT 1 FROM
                WHERE
                    @DeConta
                AND
                    @Valor

        BEGIN
            RAISERROR('Saldo insuficiente', 16, 1)
            ROLLBACK
            RETURN
        END

        -- Débito
        UPDATE
            SET
                @Valor
            WHERE
                @DeConta

        -- Crédito
        UPDATE
            SET
                @Valor
            WHERE
                @ParaConta

        -- Auditoria
        INSERT INTO

        VALUES
            @DeConta @ParaConta @Valor GETDATE

        COMMIT
        SET @Sucesso = 1
    END TRY
    BEGIN CATCH
        IF XACT_STATE() = 0 ROLLBACK
        -- Log erro...
    END CATCH
END
GO
```

✓ Procedure criada · dbo.usp_TransferirFundos

ACID · OUTPUT @Sucesso

PROGRAMABILIDADE

COMPARATIVO

Funções Definidas pelo Usuário (UDFs)

Três categorias com casos de uso distintos.



01



Scalar Function

ESCALAR

DEFINIÇÃO

Retorna um único valor. Usada em **SELECT**, **WHERE**, **computed columns**.

QUANDO USAR

Cálculos, formatação, validações escalares.

PERFORMANCE

⚠ Pode degradar **SELECTs** em grandes tabelas.

```
CREATE FUNCTION
    @Valor DECIMAL 10 2
RETURNS DECIMAL 10 2
AS
BEGIN
    RETURN @Valor 0.10
END
```



02



Inline Table-Valued

INLINE

DEFINIÇÃO

Retorna **TABLE** como um único **SELECT**. Performance similar a view parametrizada.

QUANDO USAR

Consultas filtradas reutilizáveis.

PERFORMANCE

✓ Excelente — otimizador expande como view.

```
CREATE FUNCTION
    @Cidade VARCHAR 50
RETURNS TABLE
AS
RETURN
    SELECT
    FROM
    WHERE @Cidade
```



03



Multi-statement TVF

MULTI-STMT

DEFINIÇÃO

Retorna **TABLE** com corpo procedural (**BEGIN/END**). Permite lógica complexa.

QUANDO USAR

Lógica intermediária antes de retornar tabela.

PERFORMANCE

⚠ Plano ruim — otimizador trata como black box.

```
CREATE FUNCTION
    @Quantidade INT
RETURNS @Result TABLE
    (INT DECIMAL)
AS
BEGIN
    INSERT INTO @Result
    SELECT TOP @Quantidade
        SUM
    FROM
        GROUP BY
    RETURN
END
```

PROGRAMABILIDADE

Triggers — DML e DDL

Triggers são procedures especiais executadas automaticamente em resposta a resposta a eventos DML (INSERT/UPDATE/DELETE) ou DDL (CREATE/ALTER/DROP). (CREATE/ALTER/DROP). Vivem no contexto da transação que as disparou.

TIPOS

DML AFTER (após INSERT/UPDATE/DELETE) ou INSTEAD OF (substituem a (substituem a operação)

DDL respondem a CREATE/ALTER/DROP em nível de banco ou servidor

LOGON disparam em conexões

TABELAS ESPECIAIS

→ `inserted` — novos valores (INSERT/UPDATE)

← `deleted` — valores antigos (DELETE/UPDATE)

BOAS PRÁTICAS

- ⚠ Evitar lógica pesada (impacta cada DML)
- ⚠ Documentar existência (invisíveis no app)
- ⚠ Preferir constraints/checks quando possível

triggers_auditoria.sql

```
-- Trigger AFTER para auditoria de atualizações
CREATE TRIGGER
ON
AFTER UPDATE
AS
BEGIN
    SET NOCOUNT ON
    INSERT INTO

        SELECT

            'Email'

            SUSER_SNAME
            GETDATE
        FROM
        INNER JOIN            ON
        WHERE

END
GO

-- DDL Trigger para auditoria de esquema
CREATE TRIGGER
ON DATABASE
FOR
AS
BEGIN
    INSERT INTO
    VALUES
        EVENTDATA    value$/EVENT_INSTANCE/EventType)[1]'
                    'nvarchar(100)'
        EVENTDATA    value$/EVENT_INSTANCE/TSQLCommand)[1]'
                    'nvarchar(max)'
        SUSER_SNAME
        GETDATE

END
GO
```

ROBUSTEZ

Tratamento de Erros e Transações

T-SQL moderno (SQL Server 2005+) implementa TRY/CATCH estruturado, similar a similar a linguagens como C# e Java. Combinado com transações explícitas, permite explícitas, permite garantir atomicidade mesmo em cenários de falha.

FUNÇÕES DE ERRO NO CATCH

ERROR_NUMBER()	número do erro
ERROR_SEVERITY()	severidade (0-25)
ERROR_MESSAGE()	mensagem
ERROR_LINE()	linha
ERROR_PROCEDURE()	procedure
ERROR_STATE()	estado interno

ESTADOS DE TRANSAÇÃO

XACT_STATE() = 1	commitável
XACT_STATE() = 0	não ativa
XACT_STATE() = -1	uncommittable (deve ROLLBACK)

● ● ● usp_AtualizarEstoque.sql

```
-- Padrão robusto: TRY/CATCH + transação
CREATE PROCEDURE
    @ProdutoId INT
    @Quantidade INT
AS
BEGIN
    SET NOCOUNT ON
    SET XACT_ABORT ON -- auto-rollback em erro grave
    BEGIN TRY
        BEGIN TRAN
        UPDATE
            @Quantidade
        WHERE
            @ProdutoId
        IF @@ROWCOUNT = 0
            THROW 50000 'Produto não encontrado' 1
        INSERT INTO

        VALUES @ProdutoId @Quantidade GETDATE
        COMMIT
    END TRY
    BEGIN CATCH
        -- Se há transação ativa, faz rollback
        IF XACT_STATE = 0
            ROLLBACK TRAN
        -- Log estruturado
        INSERT INTO

        VALUES
            ERROR_NUMBER    ERROR_SEVERITY
            ERROR_STATE      ERROR_MESSAGE
            ERROR_PROCEDURE  ERROR_LINE
            GETDATE
        -- Re-lança o erro para o cliente
        THROW
    END CATCH
END
GO
```

CONSULTAS AVANÇADAS

CTEs, Window Functions e Recursão

Common Table Expressions (CTEs) são result sets nomeados e temporários, temporários, definidos na cláusula WITH. Permitem recursão, legibilidade e reutilização de subconsultas.

RECURSOS CORRELATOS

- 📊 WINDOW FUNCTIONS: ROW_NUMBER, RANK, DENSE_RANK, DENSE_RANK, LAG, LEAD, SUM OVER
- → PIVOT / UNPIVOT — transformação de linhas em colunas
- ☎️ CROSS APPLY / OUTER APPLY — chamada de função por linha
- ♻️ Recursão — hierarquias (organograma, BOM)
- Múltiplas CTEs em um WITH

cte_recursiva_window.sql

```
-- CTE recursiva: hierarquia de funcionários
WITH RECURSIVE AS
-- Âncora: presidente (sem chefe)
SELECT 1 AS
FROM
WHERE IS NULL
UNION ALL
-- Recursivo: subordinados
SELECT + 1
FROM
INNER JOIN ON =

SELECT FROM
WHERE <= 3 ORDER BY ;

-- Window Function: ranking por departamento
WITH AS
SELECT
    ROW_NUMBER OVER PARTITION BY ORDER BY DESC AS
    -LAG OVER PARTITION BY ORDER BY DESC AS
    SUM OVER PARTITION BY AS
FROM

SELECT * FROM WHERE <= 3
ORDER BY ;
```

✓ CTE + Window Function · 2 blocos

Ln 24 · UTF-8

BLOCO 03 / 03

03

Ambiente, Monitoramento e Otimização de Desempenho

Ferramentas, técnicas e boas práticas que diferenciam um DBA júnior de um sênior.

- 01 → SQL Server Management Studio (SSMS)
- 02 → Ferramentas de Monitoramento
- 03 → Índices e Estatísticas
- 04 → Plano de Execução
- 05 → Boas Práticas de Desempenho

IDE OFICIAL · MICROSOFT

SQL Server Management Studio (SSMS)

IDE oficial para administração, desenvolvimento e diagnóstico do SQL Server.



01

Object Explorer

Navegação hierárquica por bancos, tabelas, views, procedures, índices. Permite gerar scripts automaticamente.



02

Query Editor

Editor T-SQL com IntelliSense, coloração de sintaxe e execução passo-a-passo (debug). Mostra plano de execução gráfico.



03

Templates

Biblioteca de templates T-SQL (criar procedures, índices, backups). Acesso via `Ctrl+Alt+T`.



04

Activity Monitor

Monitor em tempo real: processos, waits, I/O, performance. Atalho: `Ctrl+Alt+A`.



05

Execution Plan

Plano de execução gráfico. Identifica custos, scans vs seeks, operadores paralelos. Estimado (`F5`) e Real (`Ctrl+M`).



06

Profiler / XEvent

Captura de eventos: queries executadas, deadlocks, blocking. Profiler legado; preferir Extended Events.

MONITORAMENTO

Ferramentas de Monitoramento

Cada ferramenta tem papel específico na diagnostics toolkit do SQL Server.

ASPECTO	SQL Profiler	Extended Events	Query Store	DMVs
TIPO	Trace legado	Eventos leves	Histórico de queries	Visões de sistema
CUSTO	⚠ Alto overhead	✓ Muito baixo	✓ Baixo	✓ Nulo
PERSISTÊNCIA	Arquivo trace/tabela	Arquivo/anel buffer	Banco interno	Em memória
FOCO	Captura de eventos	Captura granular	Performance queries	Estado do servidor
RECOMENDADO	— Não (deprecated)	✓ Sim (substituto Profiler)	✓ Sempre ON	✓ Diagnóstico ad-hoc
DISPONÍVEL DESDE	SQL 7.0 (1998)	SQL 2008	SQL 2016	SQL 2005
CASO DE USO	Legado	Deadlocks, waits	Top queries, regress	Estado atual do servidor

◆ DICA: habilitar Query Store em todo banco de produção — `ALTER DATABASE ... SET QUERY_STORE = ON`

OTIMIZAÇÃO

Índices e Estatísticas

Índices são estruturas auxiliares (B-Tree ou Columnstore) que aceleram consultas. Estatísticas são histogramas de distribuição de valores que guiam decisões do otimizador. Sem estatísticas atualizadas, o otimizador gera planos ruins.

TIPOS DE ÍNDICE

- →Clustered — define ordem física da tabela (1 por tabela)
- →Nonclustered — estrutura separada, ponteiros para dados
- →Unique — garante unicidade
- →Filtered — indexa subconjunto (WHERE)
- →Columnstore — orientado a colunas (OLAP)
- →Full-text — busca textual

COMANDOS CHAVE

CREATE INDEX · ALTER INDEX ... REBUILD/REORGANIZE

UPDATE STATISTICS · EXEC sp_updatestats

DBCC SHOW_STATISTICS

indices_estatisticas.sql

```
-- Criação de índice com INCLUDE
CREATE NONCLUSTERED INDEX
ON
INCLUDE
;
GO

-- Índice filtrado (apenas clientes ativos)
CREATE NONCLUSTERED INDEX
ON
WHERE
= 'ATIVO';
GO

-- Verificar fragmentação
SELECT
AS
AS
AS
FROM dm_db_index_physical_stats
DB_ID NULL NULL NULL 'LIMITED'
JOIN
ON
AND
=
WHERE
>10
ORDER BY
DESC
GO

-- Manutenção: rebuild (offline) ou reorganize (online)
ALTER INDEX
ON
REBUILD WITH
=ON ;
```

✓ 4 blocos · manutenção de índices

Ln 32 · UTF-8

DIAGNÓSTICO

Plano de Execução

O Query Optimizer gera um plano de execução para cada consulta, consulta, escolhendo a estratégia mais eficiente baseada em estatísticas, estatísticas, índices e custos estimados. Ler planos é skill essencial.

COMO VISUALIZAR

- › Estimado: `Ctrl+L` (sem executar)
- › Real: `Ctrl+M` (executa e mostra)
- › Texto: `SET SHOWPLAN_TEXT ON`
- › XML: `SET SHOWPLAN_XML ON`

OPERADORES A OBSERVAR

- › `Table Scan / Clustered Index Scan` — geralmente problemático
- › `Index Seek / Clustered Index Seek` — eficiente
- › `Key Lookup` — pode ser otimizado com `INCLUDE`
- › `Sort` — se evitável via índice, melhor
- › `Hash Match` — útil em grandes conjuntos
- › `Spool` — indica recomputação

MÉTRICAS

- › `SubtreeCost` · `EstimadoRows` · `ActualRows`
- › Estimado vs Real — discrepância = estatística ruim

plano_execucao.sql

```
-- Habilitar plano de execução real
SET STATISTICS IO TIMEON;
GO

-- Consulta para análise
SELECT
    COUNT      AS
    SUM        AS
FROM
    INNER JOIN      ON      =
WHERE
    AND            = 'ATIVO'
GROUP BY
HAVING SUM        >5000
ORDER BY
DESC
GO

-- Forçar uso de índice específico
SELECT *
FROM      WITH INDEX
WHERE      =42;

-- Dica de otimização via hint
SELECT TOP 100 *
FROM
WHERE      = '2026-08-01'
OPTION  OPTIMIZE FOR UNKNOWN ;
```

} Plano + Hints · 3 blocos

Ln 27 · UTF-8



Dicas de otimização (hints) devem ser último recurso — sempre que possível, confiar no otimizador.

Boas Práticas de Desempenho

Receitas validadas em produção para manter seu SQL Server saudável.

01



SELECT Orientado a Conjunto

Evite cursores e loops. Use JOIN, CTE e window functions. SQL Server é otimizado para operações em conjuntos, não linha-a-linha. Cursores podem ser 100x mais lentos.

02



SET NOCOUNT ON

Sempre em procedures. Elimina mensagens DONE_IN_PROC que consomem banda e impactam client.

03



Use Parâmetros Tipados

Evite concatenação de strings em SQL dinâmico — além de SQL Injection, causa recompilações. Use `sp_executesql` com parâmetros.

04



Manutenção de Índices

Job semanal: REBUILD em fragmentação >30%, REORGANIZE entre 5-30%. Atualizar estatísticas com FULLSCAN mensalmente.

05



SARGability

Predicados SARGable permitem uso de índice. Evitar: `WHERE YEAR(data)=2026`. Preferir: `WHERE data >= '2026-01-01' AND data < '2027-01-01'`.

06



Habilitar Query Store

Em todo banco de produção. Captura histórico de planos, permite forçar planos antigos via Query Store Hints. Default ON em SQL 2016+.

◆ Regra de ouro: meça antes de otimizar. 80% dos ganhos vêm de 20% das queries (Princípio de Pareto).

SÍNTESE DA AULA

O que vimos hoje

01 Visão Geral do SQL Server e T-SQL

Arquitetura (Relational/Storage Engine, Buffer Pool, Transaction Log), edições (Express → Enterprise) e categorias do T-SQL (DDL/DML/DCL/DTL + procedural).

02 Fundamentos da Programação T-SQL

Variáveis, controle de fluxo, batches, stored procedures, funções (scalar, inline, multi-statement), statement), triggers (DML/DDDL), tratamento de erros TRY/CATCH e CTEs recursivas com window com window functions.

03 Ambiente, Monitoramento e Otimização

SSMS como IDE principal, Query Store / Extended Events / DMVs para diagnóstico, índices índices (clustered/nonclustered/filtered), planos de execução e boas práticas consolidadas. consolidadas.

→ Próximos Passos

- Reproduzir todos os códigos no SSMS local (SQL Server Developer Edition)
- Criar base de testes [AdventureWorks2019 — gratuito]
- Habilitar Query Store em todas as DBs
- Praticar leitura de planos de execução reais
- Estudar para certificação DP-300 (Azure Database Administrator)

proxima_aula.sql

SQL

```
-- Para a próxima aula:  
USE ;  
SELECT TOP 10 * FROM ;
```

Aula 02: Tópicos Avançados — In-Memory OLTP, AlwaysOn, Segurança e TDE